

山西大学

课 程 设 计

2021-2022 第二学期

学 院	自动化与软件学院
专 业	测控技术与仪器
学生姓名	吕紫阳
学 号	201902502220
课程名称	测控系统设计与应用实习
设计题目	STM32 单片机温度测控系统
指导教师	黄继

2022 年 6 月 13 日

一、课程设计任务书

1. 设计目的

通过本课程设计巩固《测控系统设计与应用》课程教学成果；加深对测控系统的结构、原理、特性的理解；初步了解简单测控系统电路的分析方法和工程设计方法，了解与课题有关的电子线路以及元器件工程规范，正确绘制电路图，正确连接电路；初步培养良好的测控系统设计运用和相关电路设计、焊接和调试的方法，锻炼学生的实际动手能力；培养学生查阅文献资料和按课程设计任务书要求正确撰写设计文档的能力。

2. 设计任务和要求

以 STM32F103ZET6 单片机为主控芯片，选用 DS18B20 温度传感器、设计相关电路，编写对应程序代码。利用 STM32 单片机定时器的两路 PWM 输出功能控制分别代表风扇和电热丝的两个 LED 灯珠，用其亮度表示功率的变化，当环境温度高于设定温度时电热丝停止工作，让代表电风扇的灯珠 PWM 输出占空比升高，亮度上升，代表风扇功率增大。当环境温度低于设定温度时风扇停止工作，让代表电热丝的灯珠 PWM 输出占空比增大，亮度上升，代表电热丝功率增大。实现对温度相关参数的检测与控制，并用 LCD 屏幕实现实时温度的显示，正确接线并完成电路调试。

3. 原始资料

DMTFT-28 型 LCD 显示屏资料；

DS18B20 温度传感器传感器资料；

STM32F103ZET6 单片机及德飞莱 V2.41STM32 开发板资料。

4. 主要参考文献：

1) 《STM32 嵌入式微控制器快速上手》 陈志旺 电子工业出版社；

2) STM32F103ZET6 用户参考手册；

3) DMTFT-28 型 TFT 彩色触摸屏使用说明书 V2.0。

5. 设计成果形式及要求：

1) 课程设计报告。

2) 测控系统设计相关程序代码。

3) 电路实物。

4) 调试结果。

6. 工作计划及进程（按时间顺序填写）：

6 月 7 日：查阅相关资料，掌握关键器件性能；

6 月 8 日：根据相关资料设计测控系统框图，分析大致算法功能；

6 月 9-10 日：编写单片机相关软件程序，并编译、调试；

6 月 12-13 日：调试测控系统电路、撰写报告。

指导教师签字：_____

学生签字：_____

2022 年 6 月 13 日

二、课程设计说明书

1. 摘要：（摘要应扼要叙述本设计说明书的主要内容、特点、主要成果和结论性意见，大概 200-300 字左右。）

主要内容：该课程设计主要是关于温度测控系统的课程设计的电路设计。用所学到的 STM32 单片机定时器、ADC 和 LCD 屏幕显示相关知识，设计一个可以实时测量、显示、控制的温度测控系统。

特点：在温度传感器方面使用的 DS18B20 型温度传感器，此型号传感器包含了温度信号传感、调理、AD 转换功能，传感器输出直接为数字信号，大大简化了后续的信号处理电路和相关程序代码，同时提高了温度测量精度。温度显示方面采用了一块 2.8 寸的 LCD 显示屏，LCD 显示屏最大的好处是不损伤视力，显示效果也满足要求。程序设计方面实现了 PWM 输出的变频，通过拟合函数实现了占空比与温度的线性变化。

主要成果：经过一段时间的原理框图设计、程序代码编写和调试，把温度传感器和显示屏连接到开发板上，在单片机内烧录好程序，通上电后显示屏可以显示出当前环境温度，两个 LED 灯珠可以随着温度的改变实现输出亮度的变化。

结论性意见：温度测量控制系统在人类工业生产和生活中广泛应用，从钢铁厂高炉到个人房间中的空调，都有一套温度测控系统，所以掌握设计可靠的、经济型好的温度测控系统的方法对一名测控专业的学生来说十分重要。

2. 关键词：（指出现频率最高的技术词汇，一般列 3~5 个。）

STM32 单片机、PWM 输出、LCD 显示屏、DS18B20 温度传感器

3. 绪论：（绪论应简要说明选题的目的、背景和意义。）

目的：掌握基础测控系统设计方法，熟悉设计过程；提高电子电路的理论知识及较强的实践能力；对电路器件的选型及电路形式的选择有一定的了解；学习 STM32 单片机程序的基本编写能力及基本调试能力；能够正确使用实验仪器对电路进行调试与检测。

背景：本次设计是以实践性为课题，起到巩固所学知识，加强综合能力，培养测控系统设计能力，提高实验技术，启发创新思想的效果。使用温度传感器实时检测环境温度并将数据送入单片机，当温度高于设定温度时启动电风扇降温，当温度低于设定时启动电热丝升温，并利用 PWM 输出实现功率的调节和变频。

意义：从建国以来，我国的电子技术的不断地飞速发展，广泛应用于社会的各个领域，工业、农业、医学、军事及日常生活等，但我国由于没有赶上前两次科技革命，经济发展相对于发达国家比较晚，仍属于发展中国家，所以作为 21 世纪的大学生有义务肩负起振兴国家的使命。鉴于以上各种事实，同样也点燃了我决心未来成为一名电子工程师内心的火花，我认为在学习好文化知识的基础上，我们应该多实践，把所学到的知识能尽量多地转化为造福社会的生产力。

4. 设计原理（本次课程设计中采用的基本方法或者基本原理。）

STM32F103ZET6 单片机内部包含 2 个高级控制定时器、4 个控制定时器、两个基本定时器。4 个可同步运行的通用定时器（TIM2、TIM3、TIM4 和 TIM5）中，每个定时器都有一个 16 位的自动加载递增/递减计数器、一个 16 位预分频器和 4 个独立的通道。它适用于多种场合，包括测量输入信号的脉冲长度（输入捕获），或者产生所需要的输出波形（输出比较、产生 PWM、单脉冲输出等）。

本此温度测控系统课程设计是利用温度传感器采集当前所处环境温度，并对采集回来的温度数据进行处理，最终在显示屏上显示当前环境温度。

同时当前环境温度数据会与单片机程序中设定的适宜温度进行比较，若当前环境温度高于设定的适宜温度，系统开启电风扇，关闭电热器，同时使用通用定时器的 PWM 输出功能控制输出功率。若当前环境温度低于设定的适宜温度，系统开启电热器，关闭电风扇，同时使用通用定时器的 PWM 输出功能控制输出功率。

5. 设计方案（设计的详细思路架构和主要内容。）

思路架构：

设计了采用了 DS18B20 型温度传感器，DS18B20 是美国 DALLAS 半导体公司推出的第一片支持“一线总线”接口的温度传感器，它具有微型化，低功耗，高性能，抗干扰能力强，易配微处理器等优点，可直接将温度转化成数字信号处理器处理。测量的温度范围是 -55°C ~ 125°C ，测温误差 0.5°C 。

在显示方面采用了 DMTFT-28 型号 LCD 液晶显示屏，具有寿命长、易于彩色化、低功耗、可以做到完全不发热、具有高精度的画质等优点。

采集到温度数据后利用拟合好的一元线性函数可以由温度数据得到合适的 PWM 输出的 CCR 的值，从而让电热器和电风扇的输出功率可以跟随温度线性变化。

主要内容：

温度测量与显示：

先进行硬件方面的观察与大致规划再编程，杜绝马虎大意。

1. 在显示屏上找到 VCC 和 GND 引脚，查阅显示屏说明书，查看 VCC 的电压，给显示屏正确供电和接地。

2. 使用厂家提供的 DMTFT-28 相关程序代码，找到适合 STM32F103ZET6 型号单片机的程序，在编程时选择开启开发板上合适的 I/O 口，让数据和控制信号顺利传输给 LCD 显示屏。

3. 在厂商配套的资料中找到字库和显示字符库相关头文件和库函数文件，在编写代码时导入。否则将不能正确显示。

温度控制：

1. 首先使用 IF 判断结构判断当前温度低于还是高于设定的适宜温度，选择开启电热器还是电风扇。

2. 根据实时温度用一元线性函数得到 PWM 输出的 CCR 的值，让占空比实现变化，从而让输出功率与温度实现线性变化，即线性变频。

6. 设计内容实现与测试（根据学科背景，对设计进行公式性的演算或程序的运行界面或可实现的说明等。）

设计内容实现：

首先需要用杜邦线连接显示屏的 VCC 和 GND 引脚到开发板上的电源和地线。STM32 一般供电为 3.3V 供电，因此可能用户自行购买或者自行研发的多为 3.3V 系统，如果供电 VIN 脚也为 3.3V，那么屏幕背面的 J2 需要短接。否则 VCC 应为 5V。

然后连接屏幕的控制线和数据接口到开发板，实现显示内容的传输。根据屏幕说明书上的介绍，RS 控制线接 PA0，RW 控制线接 PA1，RD 控制线接 PA9，CS 控制线接 PB0，RST 控制线接 PB1，LE 拓展控制线接 PB2，数据接口 D10-D17 接 PC0-PC7，NC 管脚为空管脚。这就完成了显示功能的连接，可以向单片机中烧录屏幕测试程序来测试一下屏幕能否正常工作。屏幕显示功能一切正常的话就可以编写 DS18B20 温度传感器的相关程序代码，此传感器程序实例随单片机开发板附带，在其上面加入显示部分程序即可，开发板上带有 18B20 传感器专用接口，但是在连接时一定要注意三根管脚的连接是否正确，接地管脚如果和电源管脚接反的话会使传感器烧毁。正确连接并且烧录好程序后测试一下屏幕能否正确显示温度，可以正确显示的话则进行下一步工作，不能正常工作的话检查代码和接线是否存在问题。

下一步进行电热器和电风扇功能的实现工作。首先设定适宜温度为 25℃，使用一个 while 无限循环结构，把温度显示相关程序也放入这个循环结构中，确保可以一直运行。接下来使用 if 判断结构来判断当前温度是否高于或者低于设定的适宜温度，若高于适宜温度，开启 I/O 口 PB5，配置为复用输出模式，并且使能。使能 TIM3 通用定时计数周期 per 设置为 900，让 TIM3_CH2 部分重映射到 PB5 上，开启此通道的 PWM 输出功能，利用拟合好的一元线性函数得到温度值与 CCR 的关系，温度越高，CCR 的值越大，但是最大不能超过 900。当温度低于适宜温度 25℃时，开启 I/O 口 PE5，配置为复用输出模式，查阅 STM32 单片机参考手册查找与 PE5 相连接的定时器以及具体通道，其余步骤与上文中的 PWM 输出配置方法相似。下图为拟合的 CCR 与温度的一元线性函数关系。

```
led0pwmval=13*temperature-3262;
```

测试：

首先测试屏幕的显示功能是否正常，首先利用电吹风对传感器所处环境适当加热，观察屏幕显示的变化，观察温度示数准确，没有乱码情况发生。同时升温超过设定适宜温度后观察 PB5 上连接的 LED 灯点亮，随着温度的上升亮度发生变化，亮度提高。接下来让温度测控系统所处环境温度适当下降，观察屏幕示数和 PE5 上连接的 LED 灯正常点亮，随着温度的下降 LED 灯的亮度上升。即温度测控系统工作正常，达到设计要求。

7. 不足与缺点（总结本设计存在的不足之处、缺点或未来可改进之处。）

本设计过程中，在查看电路器件时准备工作不够充分，在连线过程中一些器件连接时需要注意正负极问题，没有及时了解马马虎虎，拖延了进度，从调试结果看，连线时犹豫马虎导致调试结果不太理想，最终结果与理想结果不符，经过和同学老师们的一起检查电路，发现有一处连接点不牢固，最后连接后实现了理想的结果，但是在高温环境下，它的PB5上的LED灯不会亮，这个问题经过探索，发现时在编程阶段出了问题，没有正确初始化结构体，改正后问题得到解决。本设计的缺点是焊点太多，元器件复杂种类丰富，只要不细心一秒就有可能实现不了预期结果，但是通过我的不懈精神，刻苦钻研，最终还是发现问题，解决了问题。通过这个设计我认为未来我可以学习这次设计的精神，不会半途而废，而会一直坚持探索问题，解决问题后的喜悦是难以掩饰的，我会发扬这次设计过程中的精神。

8. 分析与总结（对整个课程设计说明书主要成果的归纳，要突出本次课程设计的要点或创新点，以简练的文字对本次课程设计的主要工作进行评价。）

本次课程设计给我们带来了许多的新知识，增强了我们的动手能力，给生活增添了快乐，他同时也告诉我们应该与互相帮助、相互团结的团队精神。就此产品，是对我们半年来所学的知识进行全面的总结，明了自己学到了什么，还缺少什么，找到自己的不足之处，对我们以后的工作有很大的帮助。课程设计是我们学习过程中一个十分重要的环节。是锻炼我们运用所学知识正确分析和解决实际问题的重要方面，也是培养应用型专门人才的要求。此次设计，我们仅仅做出的是一个小产品，简单实用，不但大大提高了我们的动手能力，还对于我们以后开发新的产品打下了一个基础，也给予我们一个启发，面临着就业，我将充分发挥我的主观能动性和在学校学到的一切知识。为母校添光加瓦，为自己的前程奋斗！这次课程设计的顺利进行，我深刻明白了理论知识与社会实践相结合的道理，从中得到了以前书本知识所不曾得到的知识。更加明白了如今信息时代电子技能知识的重要性。增强了我对测控系统设计、实际工艺技术、电子技术和设备技术等方面的认识，掌握了分析处理方法，调试、计算等基本技能的训练，具有一定程度的实际工作能力。面对如此激烈的市场竞争体系，只有努力掌握好电子技能知识方可在竞争中立于不败之地，我对从事电子产品的开发和研究充满了兴趣。希望自己以后通过自己的不断的努力获得更多的工艺和电子技术。

9. 参考文献（要注重引用近期发表的与本课程设计直接有关的学术期刊类文献。）

- [1]智能温度传感器 DS18B20 的原理与应用 赵海兰，赵祥伟 - 《现代电子技术》2021.
- [2]由 DS18B20 构成的多点温度测量系统 蒋鸿宇，王勇，植涌 - 《单片机与嵌入式系统应用》2007.
- [3]基于 STC 单片机和无线数传模块的粮仓温湿度监控系统 伦翠芬，张小芹，李艳萍， - 《粮油加工(电子版)》2008.

三、 成 绩 评 定 表

序号	评分项目（百分制）	满分	实得分
1	学习态度、考勤		
2	设计分析的合理性、可行性、创造性		
3	设计成果质量		
4	设计验收		

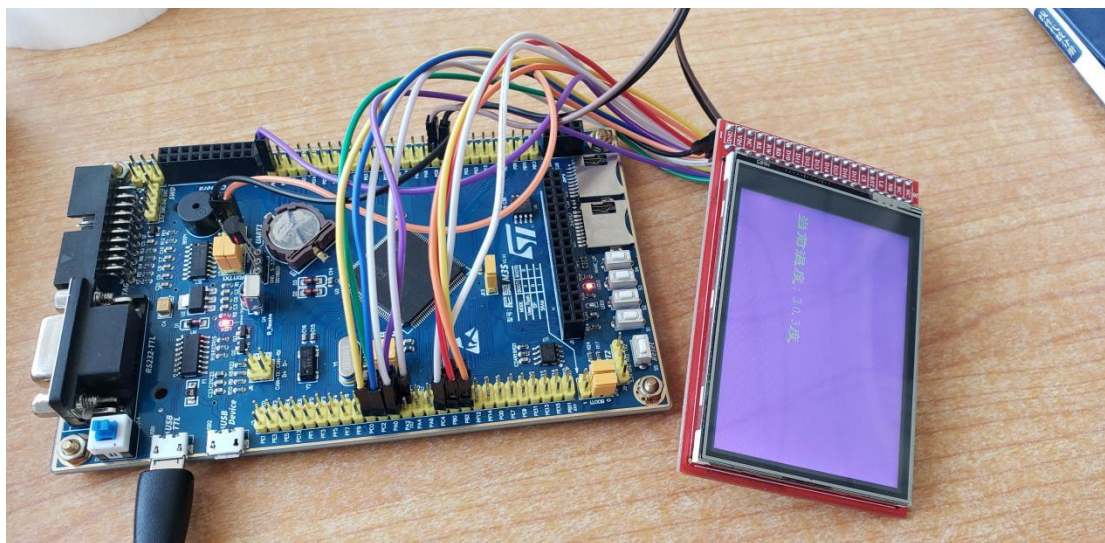
评语意见：

指导教师：_____

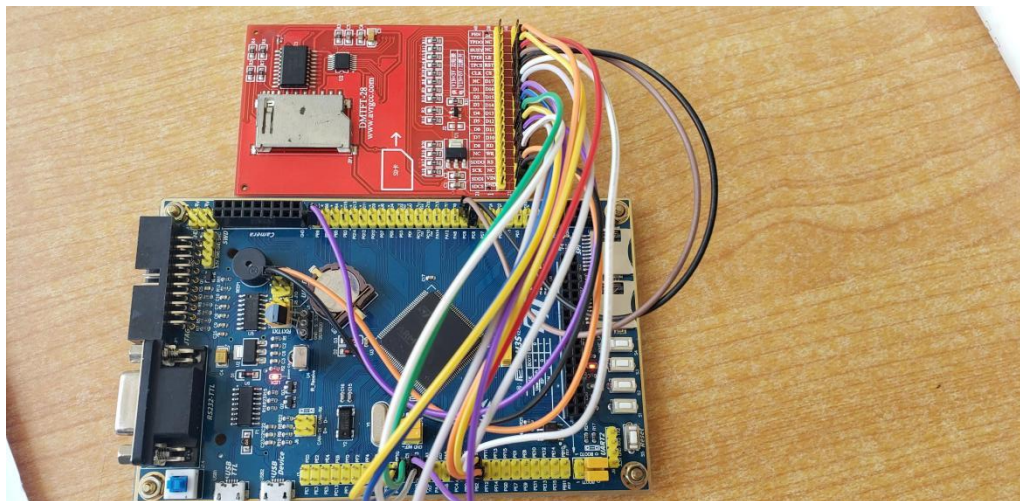
年 月 日

四、附录 (程序代码、设计图纸、实物照片等)

1. 测试过程 屏幕可以正确显示温度



2. 屏幕接线图



3. 温度高于设定适宜温度，电风扇工作



温度升高，亮度上升



4. 温度低于设定适宜温度，电热器工作



5. 重要程序

```
1 #include "18b20.h"
2 #include "delay.h"
3 //复位DS18B20
4 void DS18B20_Rst(void)
5 {
6     DS18B20_IO_OUT(); //SET P0.1 OUTPUT
7     GPIO_ResetBits(GPIOF,GPIO_Pin_11); //拉低DQ
8     delay_us(750); //拉低750us
9     GPIO_SetBits(GPIOF,GPIO_Pin_11); //DQ=1
10    delay_us(15); //150S
11 }
12 //等待DS18B20的回应
13 //返回1:未检测到DS18B20的存在
14 //返回0:存在
15 uint8_t DS18B20_Check(void)
16 {
17     uint8_t retry=0;
18     //uint16_t i;
19     DS18B20_IO_IN(); //SET P0.1 INPUT
20     //i=GPIO->IDR;
21     //i=(GPIO->IDR)&(GPIO_Pin_11);
22     while ((GPIO->IDR&0x800)&&retry<200)
23     {
24         retry++;
25         delay_us(1);
26     };
27     if(retry>200)return 1;
28     else retry=0;
29     while (!(GPIO->IDR&0x800)&&retry<240)
30     {
31         retry++;
32         delay_us(1);
33     };
34     if(retry==240)return 1;
35     return 0;
36 }
37 uint8_t DS18B20_Read_Bit(void)
38 {
39     uint8_t data;
40     DS18B20_IO_OUT(); //SET P0.1 OUTPUT
41     GPIO_ResetBits(GPIOF,GPIO_Pin_11);
42     delay_us(2);
43     GPIO_SetBits(GPIOF,GPIO_Pin_11);
44     DS18B20_IO_IN(); //SET P0.1 INPUT
45     delay_us(12);
46     if((GPIO->IDR&0x800))data=1;
```

```
18b20.c
45 delay_us(12);
46 if((GPIOC->IDR&(0x800))data=1;
47 else data=0;
48 delay_us(50);
49 return data;
50 }
51 //从DS18B20读取一个字节
52 //返回值: 读到的数据
53 uint8_t DS18B20_Read_Byte(void)
54 {
55     uint8_t i,j,dat;
56     dat=0;
57     for (i=1;i<=8;i++)
58     {
59         j=DS18B20_Read_Bit();
60         dat=(j<<7)|(dat>>1);
61     }
62     return dat;
63 }
64 //写一个字节到DS18B20
65 //dat: 要写入的字节
66 void DS18B20_Write_Byte(uint8_t dat)
67 {
68     uint8_t j;
69     uint8_t testb;
70     DS18B20_IO_OUT(); //SET P0.11 OUTPUT;
71     for (j=1;j<=8;j++)
72     {
73         testb=dat<0x01;
74         dat=dat>>1;
75         if (testb)
76         {
77             GPIO_ResetBits(GPIOC,GPIO_Pin_11); // Write 1
78             delay_us(2);
79             GPIO_SetBits(GPIOC,GPIO_Pin_11);
80             delay_us(60);
81         }
82         else
83         {
84             GPIO_ResetBits(GPIOC,GPIO_Pin_11); // Write 0
85             delay_us(60);
86             GPIO_SetBits(GPIOC,GPIO_Pin_11);
87             delay_us(2);
88         }
89     }
90 }
```

```
18b20.c
105 {
106     GPIO_InitTypeDef GPIO_InitStructure;
107
108     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE); //使能PORTG口时钟
109
110     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11; //PORTG.11 推挽输出
111     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
112     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
113     GPIO_Init(GPIOC, &GPIO_InitStructure);
114
115     GPIO_SetBits(GPIOC,GPIO_Pin_11); //输出1
116
117     DS18B20_Rst();
118
119     return DS18B20_Check();
120 }
121 //从ds18b20得到温度值
122 //精度: 0.1C
123 //返回值: 温度值 (-550-1250)
124 short DS18B20_Get_Temp(void)
125 {
126     uint8_t temp;
127     uint8_t TL,TH;
128     short tem;
129     DS18B20_Start(); // ds1820 start convert
130     DS18B20_Rst();
131     DS18B20_Check();
132     DS18B20_Write_Byte(0xcc); // skip rom
133     DS18B20_Write_Byte(0xb0); // convert
134     TL=DS18B20_Read_Byte(); // LSB
135     TH=DS18B20_Read_Byte(); // MSB
136
137     if(TH>7)
138     {
139         TH=-TH;
140         TL=-TL;
141         temp=0; //温度为负
142     }else temp=1; //温度为正
143     tem=TH; //获得高八位
144     tem<<8;
145     tem+=TL; //获得低八位
146     tem=(float)tem*0.625; //转换
147     if(temp)return tem; //返回温度值
148     else return -tem;
149 }
150 }
```

(2) 显示屏相关程序

```
1 #ifndef TFT_H
2 #define TFT_H
3 #include "stdlib.h"
4 #include "stm32f10x.h"
5 #include "delay.h"
6
7 #ifndef uchar
8 #define uchar unsigned char
9 #endif
10
11 #ifndef uint
12 #define uint unsigned int
13 #endif
14
15 /*2.8寸TFT触摸屏接口定义*/
16 #define GPIO_TFT_DATA      GPIOC          //数据口 PC0-PC7
17 #define RCC_GPIO_TFT      RCC_APB2Periph_GPIOC
18
19 #define GPIO_CTRLA        GPIOA          //控制口
20 #define RCC_GPIO_CTRLA    RCC_APB2Periph_GPIOA
21 #define GPIO_CTRB        GPIOB          //控制口
22 #define RCC_GPIO_CTRB    RCC_APB2Periph_GPIOB
23
24 #define RS_CLR            GPIO_ResetBits(GPIO_CTRLA, GPIO_Pin_0) //RS置低 RS->PA0
25 #define RS_SET            GPIO_SetBits(GPIO_CTRLA, GPIO_Pin_0)  //RS置高
26
27 #define RW_CLR            GPIO_ResetBits(GPIO_CTRLA, GPIO_Pin_1) //RW置低 RW->PA1
28 #define RW_SET            GPIO_SetBits(GPIO_CTRLA, GPIO_Pin_1)  //RW置高
29
30 #define RD_CLR            GPIO_ResetBits(GPIO_CTRB, GPIO_Pin_9)  //RD置低 RD->PB9
31 #define RD_SET            GPIO_SetBits(GPIO_CTRB, GPIO_Pin_9)  //RD置高
32
33 #define CS_CLR            GPIO_ResetBits(GPIO_CTRB, GPIO_Pin_0)  //CS置低 CS->PB0
34 #define CS_SET            GPIO_SetBits(GPIO_CTRB, GPIO_Pin_0)  //CS置高
35
36 #define RST_CLR           GPIO_ResetBits(GPIO_CTRB, GPIO_Pin_1)  //RST置低 RST->PB1
37 #define RST_SET           GPIO_SetBits(GPIO_CTRB, GPIO_Pin_1)  //RST置高
38
39 #define LE_CLR            GPIO_ResetBits(GPIO_CTRB, GPIO_Pin_2)  //LE置低 LE->PB2
40 #define LE_SET            GPIO_SetBits(GPIO_CTRB, GPIO_Pin_2)  //LE置高
41
42 #define DataPort           GPIOC          //PC口为数据口
43
44 #define D0                 GPIO_Pin_0    //D0使用的GPIO引脚
45 #define D1                 GPIO_Pin_1    //D1使用的GPIO引脚
46 #define D2                 GPIO_Pin_2    //D2使用的GPIO引脚
47
48 #define DataPort           GPIOC          //PC口为数据口
49
50 #define D0                 GPIO_Pin_0    //D0使用的GPIO引脚
51 #define D1                 GPIO_Pin_1    //D1使用的GPIO引脚
52 #define D2                 GPIO_Pin_2    //D2使用的GPIO引脚
53 #define D3                 GPIO_Pin_3    //D3使用的GPIO引脚
54 #define D4                 GPIO_Pin_4    //D4使用的GPIO引脚
55 #define D5                 GPIO_Pin_5    //D5使用的GPIO引脚
56 #define D6                 GPIO_Pin_6    //D6使用的GPIO引脚
57 #define D7                 GPIO_Pin_7    //D7使用的GPIO引脚
58
59 /*全局变量-颜色代码*/
60
61 #define White              0xFFFF        //显示颜色代码
62 #define Black              0x0000
63 #define Blue              0x001F
64 #define Blue2             0x005F
65 #define Red               0xF800
66 #define Magenta           0xF01F
67 #define Green             0x07E0
68 #define Cyan              0x7FFF
69 #define Yellow            0xFFE0
70
71 extern unsigned int Device_code;
72
73 void Write_Cmd(unsigned char DH,unsigned char DL);
74 void Write_Data(unsigned char DH,unsigned char DL);
75 void Write_Cmd_Data (unsigned char x,unsigned int y);
76 void Write_Data_U16(unsigned int y);
77 void LCD_SetPos(unsigned int x0,unsigned int x1,unsigned int y0,unsigned int y1);
78 void CLR_Screen(unsigned int bColor);
79 void LCD_PutChar(unsigned int x, unsigned int y, char c, unsigned int fColor, unsigned int bColor);
80 void LCD_PutSingleChar(unsigned int x, unsigned int y, char c, unsigned int fColor, unsigned int bColor);
81 void PutGB2424(unsigned short x, unsigned short y, unsigned char g[2], unsigned int fColor,unsigned int bColor);
82 void LCD_PutString(unsigned short x, unsigned short y, unsigned char *s, unsigned int fColor, unsigned int bColor);
83 void LCD_PutString24(unsigned short x , unsigned short y, unsigned char *s, unsigned int fColor, unsigned int bColor);
84 void Show_RGB (unsigned short x0,unsigned short x1,unsigned short y0,unsigned short y1,unsigned int Color);
85 void gpio_init(void);
86 void TFT_Init(void);
87
88 #endif
```

(3) 主函数程序

```

1 #include "stm32f10x.h"
2 #include "stm32f10x_gpio.h"
3 #include "delay.h"
4 #include "gft.h"
5 #include "timer.h"
6 char *reverse(char *s);
7 char *my_itoa(int n);
8 char *reverse(char *s)
9 {
10     char temp;
11     char *p = s; //p指向s的头部
12     char *q = s; //q指向s的尾部
13     while(*q)
14         ++q;
15     q--;
16     //交换移动指针，直到p和q交叉
17     while(q > p)
18     {
19         temp = *p;
20         *p++ = *q;
21         *q-- = temp;
22     }
23     return s;
24 }
25
26 char *my_itoa(int n)
27 {
28     int i = 0, isNegative = 0;
29     static char s[100]; //必须为static变量，或者是全局变量
30     if((isNegative = n) < 0) //如果是负数，先转为正数
31     {
32         n = -n;
33     }
34     do //从各位开始变为字符，直到最高位，最后应该反转
35     {
36         s[i++] = n%10 + '0';
37         n = n/10;
38     }while(n > 0);
39     if(isNegative < 0) //如果是负数，补上负号
40     {
41         s[i++] = '-';
42     }
43     s[i] = '\0'; //最后加上字符串结束符
44     return reverse(s);
45 }
46

```

```

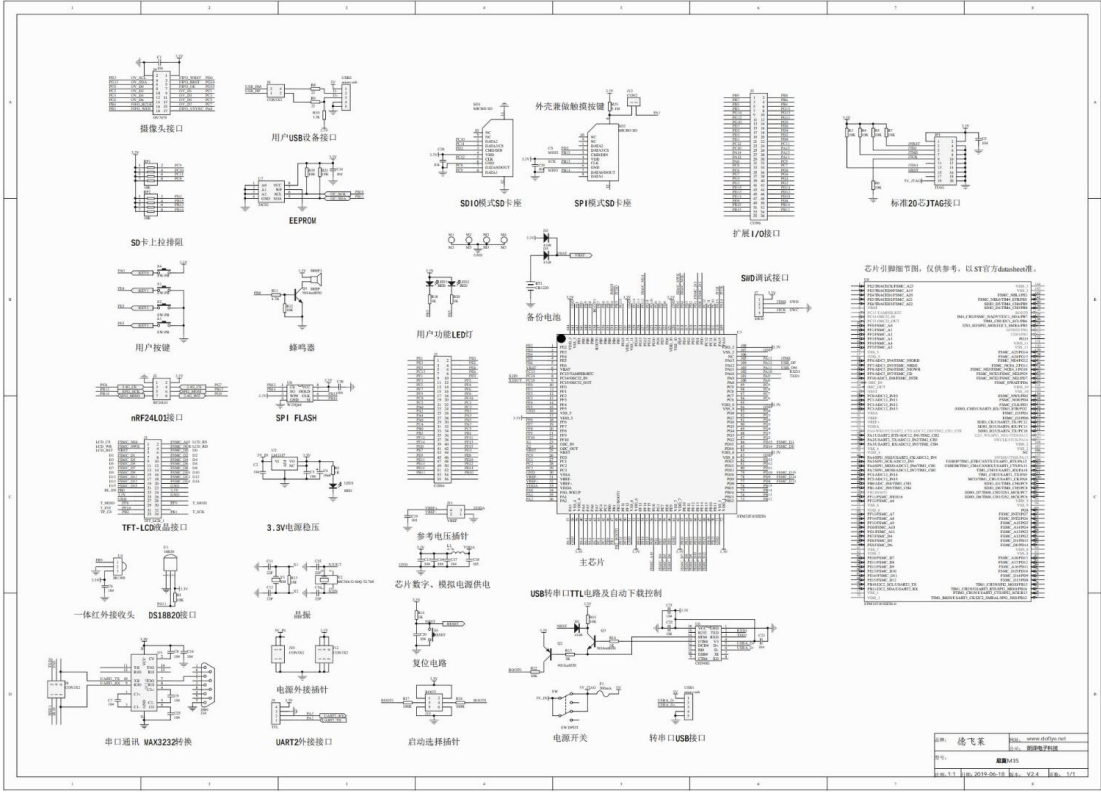
45     return reverse(s);
46 }
47 int main(void)
48 {
49     uint8_t t=0;
50     uint16_t ledOpwmval=0;
51     uint16_t ledIpwmval=0;
52     uint16_t i;
53     uint8_t dir=1;
54     short temperature,H_temperature,L_temperature;
55     char *a;
56
57     delay_init();
58     gpio_init();
59     TFT_Init();
60     DS18B20_Init();
61     delay_ms(200);
62     delay_ms(200);
63     CLR_Screen(Magenta);
64     while(1)
65     {
66         if(t%10==0) //每100ms读取一次
67         {
68             temperature=DS18B20_Get_Temp();
69             H_temperature=temperature/10;
70             L_temperature=temperature%10;
71             if(temperature<0)
72             {
73                 //LCD_ShowChar(30+40,150,'-',16,0); //显示负号
74                 //OLED_ShowString(47,24,'-',16);
75                 //OLED_Refresh_Gram();
76                 temperature=-temperature; //转为正数
77             }
78             a=my_itoa(H_temperature);
79             LCD_PutString24(10,140,"当前温度:",Yellow,Magenta);
80             LCD_PutString24(120,140,a,Yellow,Magenta);
81             LCD_PutString24(140,140,".",Yellow,Magenta);
82
83             a=my_itoa(L_temperature);
84             LCD_PutString24(148,140,a,Yellow,Magenta);
85             LCD_PutString24(156,140,"度",Yellow,Magenta);
86             // LCD_ShowChar(30+40,150,' ',16,0); //去掉负号
87             // LCD_ShowNum(30+40+8,150,temperature/10,2,16); //显示正数部分
88             // LCD_ShowNum(30+40+32,150,temperature%10,1,16); //显示小数部分
89         }
90         delay_ms(10);
91     }
92 }

```

```
main.c
90     delay_ms(10);
91     t++;
92     if(t==20)
93     {
94         t=0;
95     }
96     //CLR_Screen(Yellow); //用背景色清屏
97     // LCD_PutString24(10,140,"当前温度:",Yellow,Magenta); //欢迎界面, 24x24大小汉字, 字模软件隶书小二号
98
99
100
101     TIM3_PWM_Init(899,0);
102     GPIO_InitTypeDef GPIO_InitStructure;
103     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
104     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
105     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
106     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
107     GPIO_Init(GPIOB, &GPIO_InitStructure);
108
109     delay_ms(10);
110     if(temperature>=251&&temperature<=319)
111     {
112         ledOpwmval=13*temperature-3262;
113
114         TIM_SetCompare2(TIM3,ledOpwmval);
115     }
116 }
117 else if(temperature<240)
118 {
119     ledOpwmval=0;
120     TIM_SetCompare2(TIM3,ledOpwmval);
121     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);
122     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
123     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
124     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
125     GPIO_Init(GPIOE, &GPIO_InitStructure);
126
127 }
128 }
129 else if(temperature>320)
130 {
131     ledOpwmval=899;
132
133     TIM_SetCompare2(TIM3,ledOpwmval);
134     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);
135     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
```

```
main.c
99
100
101     TIM3_PWM_Init(899,0);
102     GPIO_InitTypeDef GPIO_InitStructure;
103     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
104     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
105     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
106     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
107     GPIO_Init(GPIOB, &GPIO_InitStructure);
108
109     delay_ms(10);
110     if(temperature>=251&&temperature<=319)
111     {
112         ledOpwmval=13*temperature-3262;
113
114         TIM_SetCompare2(TIM3,ledOpwmval);
115     }
116 }
117 else if(temperature<240)
118 {
119     ledOpwmval=0;
120     TIM_SetCompare2(TIM3,ledOpwmval);
121     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);
122     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
123     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
124     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
125     GPIO_Init(GPIOE, &GPIO_InitStructure);
126
127 }
128 }
129 else if(temperature>320)
130 {
131     ledOpwmval=899;
132
133     TIM_SetCompare2(TIM3,ledOpwmval);
134     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOE, ENABLE);
135     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
136     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
137     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AIN;
138     GPIO_Init(GPIOE, &GPIO_InitStructure);
139 }
140
141
142 }
143 }
144 }
```

6. 开发板 PCB 原理图



7. DMTFT-28 型 TFT 触摸屏转接板元器件清单

序号	类型	规格	封装	标号	数量	备注
1	IC	74HC573D	宽体 SOP20	U3	1	
2	IC	AMS1117-3.3	贴片 TO23	U1	1	
3	IC	XPT2046	贴片 SOP16	U2	1	
4	电阻	103 (10K)	0805	R3	1	
5	电阻	102 (1K)	0805	R8,R9,R10,R11,R12,R13,R14,R15,R16,R17,R18,R19,R20,R21	14	
6	电阻	101 (100)	0805	R4,R5,R6,R7	4	
7	电阻	3R3 (3.3)	0805	R2	1	
8	电容	4.7uF 钽电容	1206	C3	1	
9	电容	104 (0.1uF)	0805	C1,C2,C4,C5	4	
10	三极管	9012	贴片 TO23	Q1	1	
11	SD 座	SD 卡座	短体不自弹	JP1	1	